

COMPETENCY QUESTION 01

Name : Rahman MD Shazedur

ID:M25W7502

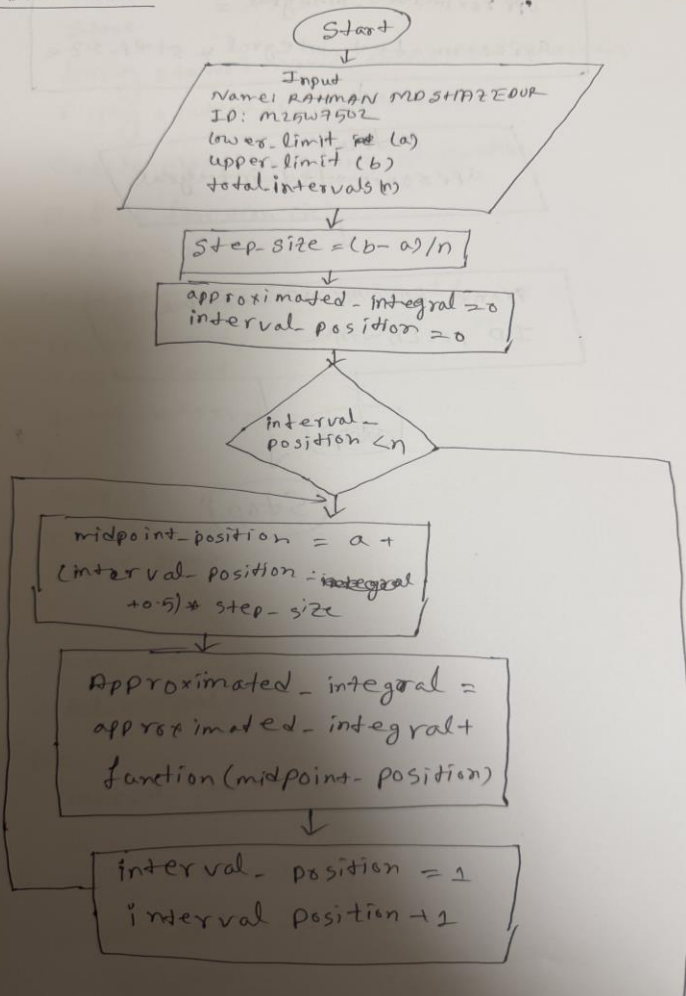
Midpoint Rule

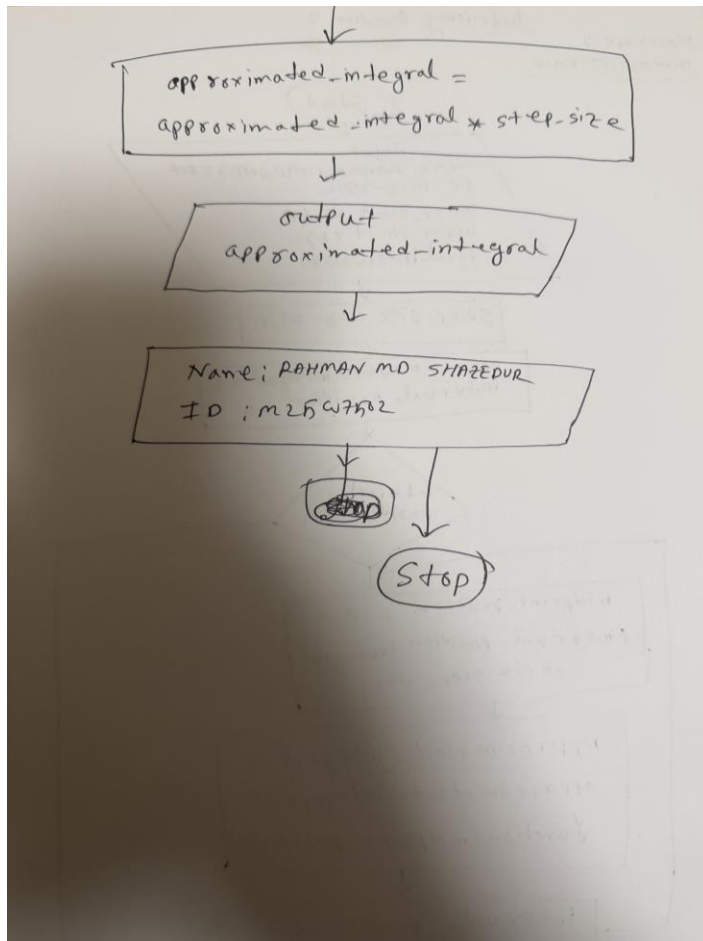
1. Flowcharts

Proficiency Question: 2

Flowchart: 2

MIDPOINT RULE





Pseudocode

Pseudocode → 1 (midpoint Rule)

```
Start
Display student name: RAHMAT MO SHARIDUR
Display student ID: M25W7502
Create class midpointIntegration
Define function (x)
Return x * x
Define calculateIntegral (
    lower_limit,
    upper_limit
    total_intervals
)
    step_size = 0
    (
        upper_limit
        -
        lower_limit
    )
    total_integral = 0
    for,
        interval_position = 0
        To
            total_intervals - 1
            midpoint_position =
            lower_limit
```

(interval-position+0.5)

x step-size

approximated-integral =

approximated-integral +

function (

midpoint-position

)

END FOR

approximated-integral =

approximated-integral

x step-size

Display.Result

Display

RATHMAN MD SHARZUQ

m25w7502

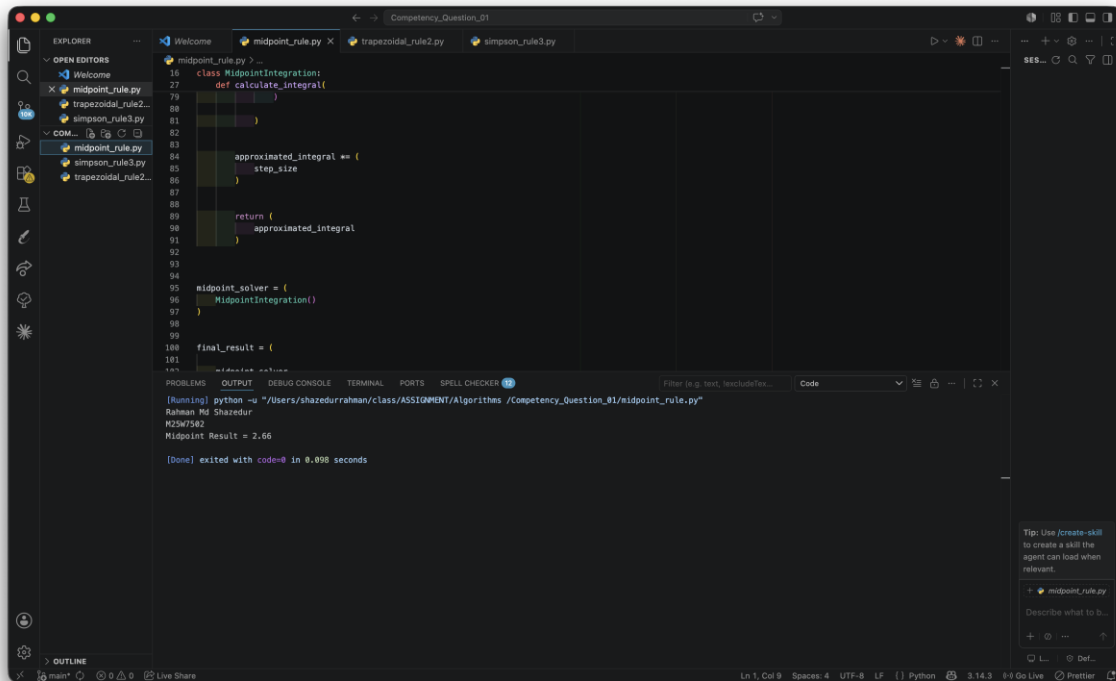
stop

Code

```
1 # Import
2 # Student Name : Rahman Md Shazedur
3 # Student ID : M25W7582
4 # Course : Fundamentals of Algorithms
5 # Program : Midpoint Rule
6 # Name
7
8
9 class StudentInformation:
10     full_name = "Rahman Md Shazedur"
11
12     student_id = "M25W7582"
13
14
15
16 class MidpointIntegration:
17
18     def mathematical_function(
19         self,
20         x
21     ):
22
23
24         return x * x
25
26     def calculate_integral(
27         self,
28         lower_limit,
29         upper_limit,
30         total_intervals
31     ):
32
33
34         step_size = (
35             upper_limit
36             -
37             lower_limit
38         ) / total_intervals
39
40         approximated_integral = 0
41
42         for interval_position in range(
43             0,
44             total_intervals,
```

```
16 class MidpointIntegration:
17     def calculate_integral(
18         self,
19         lower_limit,
20         upper_limit,
21         total_intervals
22     ):
23
24
25         step_size = (
26             upper_limit
27             -
28             lower_limit
29         ) / total_intervals
30
31         approximated_integral = 0
32
33         for interval_position in range(
34             0,
35             total_intervals,
36             step_size
37         ):
38             approximated_integral += (
39                 self.mathematical_function(
40                     lower_limit + interval_position * step_size
41                 ) * step_size
42             )
43
44         return approximated_integral
45
46
47 midpoint_solver = (
48     MidpointIntegration()
49 )
50
51 final_result = (
52     midpoint_solver
53     .calculate_integral(
54         0,
55         2,
56         10
57     )
58 )
59
60
61 print(
62     StudentInformation.full_name
63 )
64
65 print(
66     StudentInformation.student_id
67 )
68
69 print(
70     "Midpoint Result =",
71     final_result
72 )
```

output



```
16 class MidpointIntegration:
27     def calculate_integral(
28         self,
29         a: float,
30         b: float,
31         n: int,
32     ) -> float:
33         """
34         Calculate the integral of a function f(x) over the interval [a, b]
35         using the Midpoint Rule.
36         """
37         approximated_integral = 0
38         step_size = (b - a) / n
39         for i in range(n):
40             x = a + step_size * i
41             y = f(x)
42             approximated_integral += y * step_size
43         return approximated_integral
44
45 midpoint_solver = (
46     MidpointIntegration()
47 )
48
49 final_result = (
50     midpoint_solver.calculate_integral(
51         a=0,
52         b=1,
53         n=1000,
54     )
55 )
56
57 print(f"Midpoint Result = {final_result}")
```

OUTPUT

```
[Running] python -m "/Users/shazedurrahman/class/ASSIGNMENT/Algorithms /Competency_Question_01/midpoint_rule.py"
Rahman Md Shazedur
M29W582
Midpoint Result = 2.66
[Done] exited with code=0 in 0.098 seconds
```

Conclusion – Program 1

This program was completed using the Midpoint Rule method in Python.

The algorithm calculated the approximate integral value by selecting the midpoint of each interval and summing all values.

The implementation process helped me understand iteration, interval calculation, and numerical approximation.

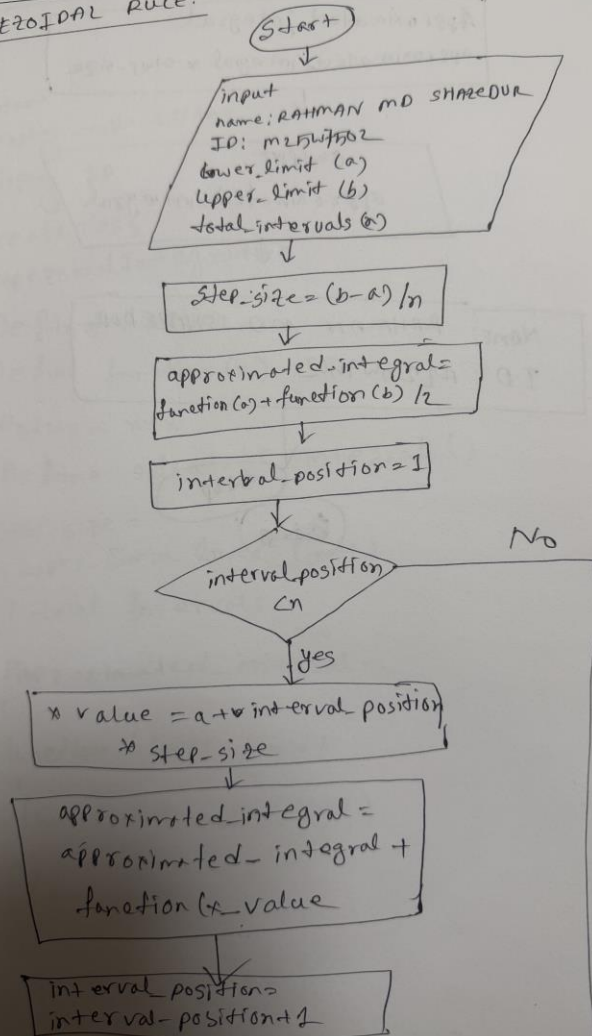
The generated output showed that the Midpoint Rule can produce a simple and efficient approximation result.

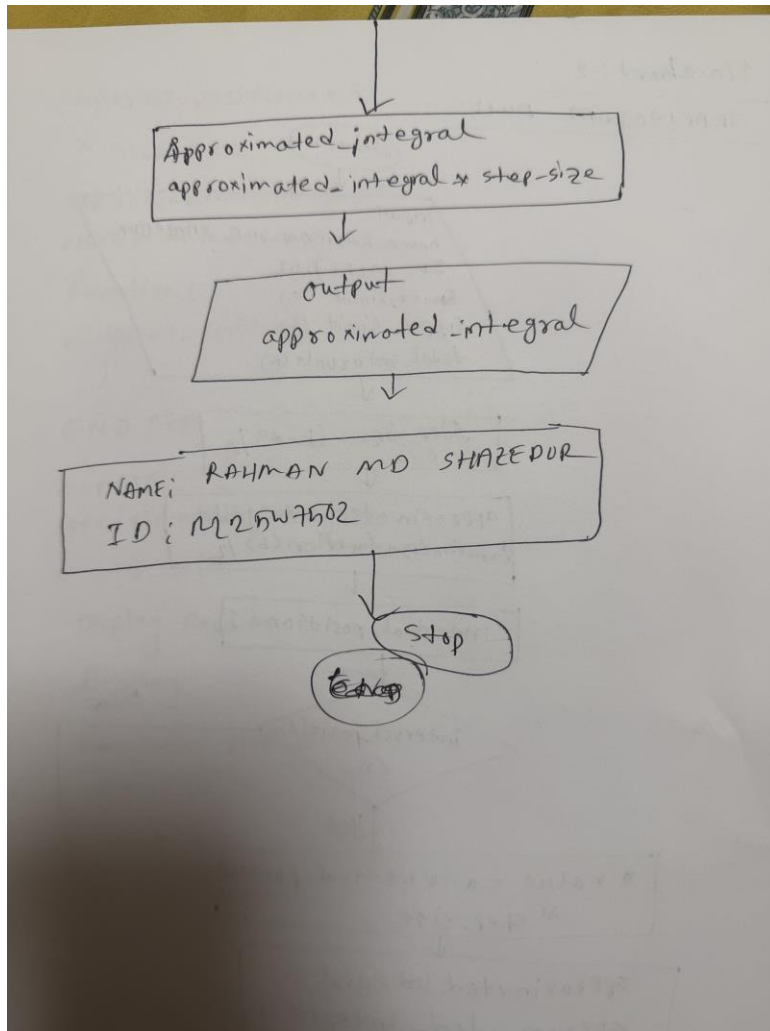
Trapezoidal Rule

1. Flowcharts

Flowchart: 2

TRAPEZOIDAL RULE:





Pseudocode

PSEUDOCODE-2 TRAPEZOIDAL RULE :

```
Start
Display studentName
Display ID
create class
TrapezoidalIntegration
Define function
Define function (n)
Return xxx
Define calculate_integral()
Step size =
(upper-limit - lower-limit)
/ total intervals
Approximated_integral =
(
function (lower-limit)
+
function (upper-limit)
)
/ 2
for .
interval position
```

= 1 To total intervals - 1

x value =

lower-limit +
interval position

x step-size

Approximated-integral =
Approximated-integral

~~x step-size~~ function(x-value)

END FOR

Approximated-integral =

Approximated-integral .
x step-size

Display Result

Display

RATHMAN MD SHAZEDUR

M2BWT502

Stop

Code

This screenshot shows the first part of a Python script in a VS Code editor. The Explorer sidebar on the left shows a project named 'Competency_Question_01' with files 'Welcome', 'midpoint_rule.py', 'trapezoidal_rule2.py', 'simpson_rule3.py', and 'trapezoidal_rule2.py'. The main editor window displays the code for 'trapezoidal_rule2.py'.

```
1 # =====
2 # Student Name : Rahman Md Shazedur
3 # Student ID : K25K7582
4 # Program : Trapezoidal Rule
5 # =====
6
7
8 class StudentInformation:
9
10     full_name = "Rahman Md Shazedur"
11
12     student_id = "K25K7582"
13
14
15 class TrapezoidalIntegration:
16
17     def mathematical_function(
18         self,
19         x
20     ):
21
22         return x * x
23
24
25     def calculate_integral(
26         self,
27         lower_limit,
28         upper_limit,
29         total_intervals
30     ):
31
32
33         step_size = (
34             upper_limit
35             -
36             lower_limit
37             ) / total_intervals
38
39
40         approximated_integral = (
41             self
42             .mathematical_function(
43                 lower_limit
44             )
45             +
46             +
47             +
48             +
49             +
50             self
```

This screenshot shows the second part of the Python script in the VS Code editor. The code continues from the previous screenshot, showing the implementation of the trapezoidal integration algorithm.

```
15 class TrapezoidalIntegration:
16     def calculate_integral(
17
18         approximated_integral += (
19
20             self
21             .mathematical_function(
22                 x_value
23             )
24         )
25
26
27         approximated_integral += (
28             step_size
29         )
30
31         return (
32             approximated_integral
33         )
34
35
36 solver = (
37     TrapezoidalIntegration()
38 )
39
40 result = (
41     solver
42     .calculate_integral(
43         0,
44         2,
45         10
46     )
47 )
48
49
50 print(
51     StudentInformation.full_name
52 )
53
54 print(
55     StudentInformation.student_id
56 )
57
58 print(
59     "Trapezoidal Result =",
60     result
61 )
```

Output

```
class TrapezoidalIntegration:
    def calculate_integral():
        approximated_integral += (
            self
            .mathematical_function(
                x_value
            )
        )
        approximated_integral += (
            step_size
        )
        return (
            approximated_integral
        )

solver = (
    TrapezoidalIntegration()
)

[Running] python -u "/Users/shazedurrahman/class/ASSIGNMENT/Algorithms /Competency_Question_01/trapezoidal_rule2.py"
Rahman Md Shazedur
MC50720
Trapezoidal Result = 2.6800000000000006

[Done] exited with code=0 in 0.04 seconds
```

Conclusion – Program 2

This program was implemented using the Trapezoidal Rule method.

The algorithm estimated the integral value by dividing the area into trapezoid sections and calculating their total contribution.

Through this implementation, I learned how iterative numerical methods improve approximation accuracy.

The final output demonstrated that the Trapezoidal Rule provides reliable and improved results.

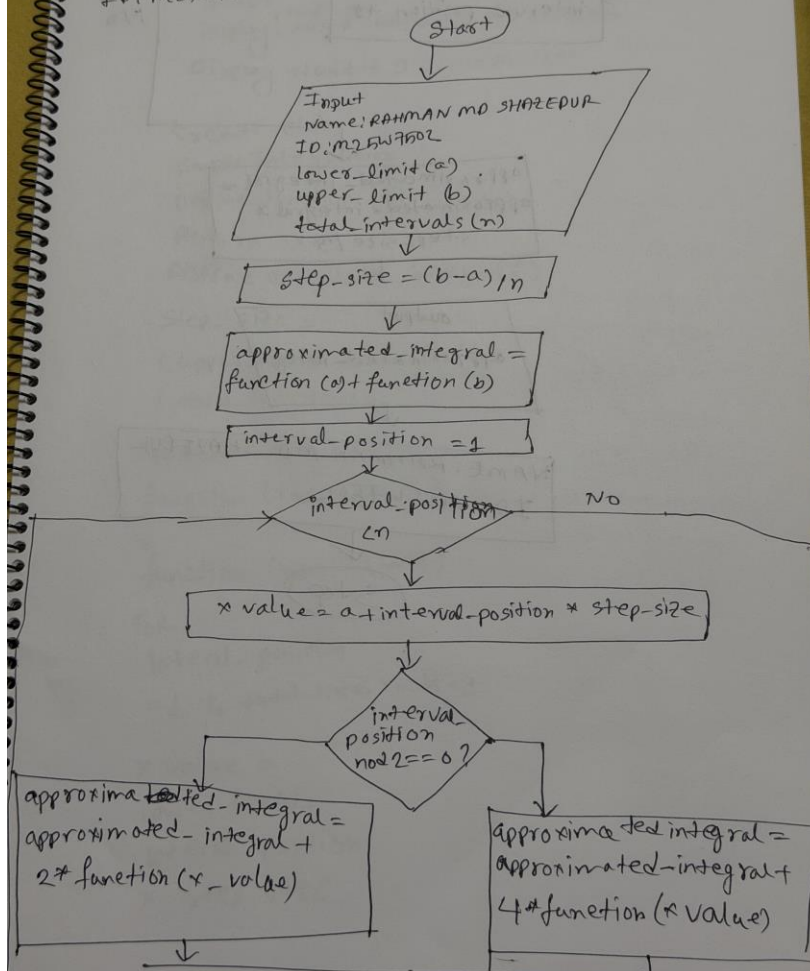
Simpson Rule

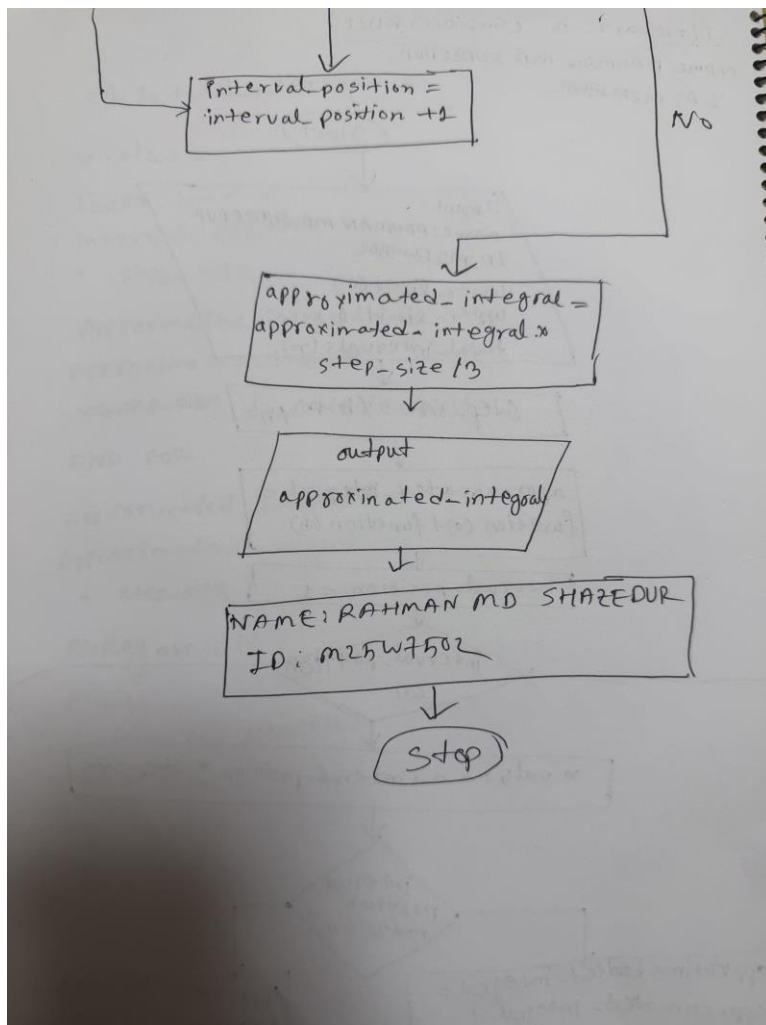
1. Flowcharts

Flowchart:-3 (Simpson's RULE)

NAME: RAHMAN MD SHARADUR

ID: M25W7502





Pseudocode

PSEUDOCODE - 3

Start student
Display Name: RAHMAN MD SHAZEDOR
Display student ID: M25W7504

Create class
simpsoIntegration

Define function (x)

Return xxx

Define calculate_integral()

Step-size =

(upper-limit - lower-limit)

/ total-intervals

approximated-integral =

function (lower-limit)

+
function (upper-limit)

For

interval-position

= 1 To total-intervals - 1

X value =

lower-limit +

interval-position

x step size

```

IF
  interval-position
  mod 2 = 0
    approximated-integral =
    approximated-integral +
    2 * function (x-value)
  ELSE
    approximated-integral =
    approximated-integral +
    4 * function (x-value)
  END IF
END FOR
approximated-integral
approximated-integral
x step-size / 3
Display Result-1
RAHMAN MD SHARADUR
m26w7502
stop

```

Code

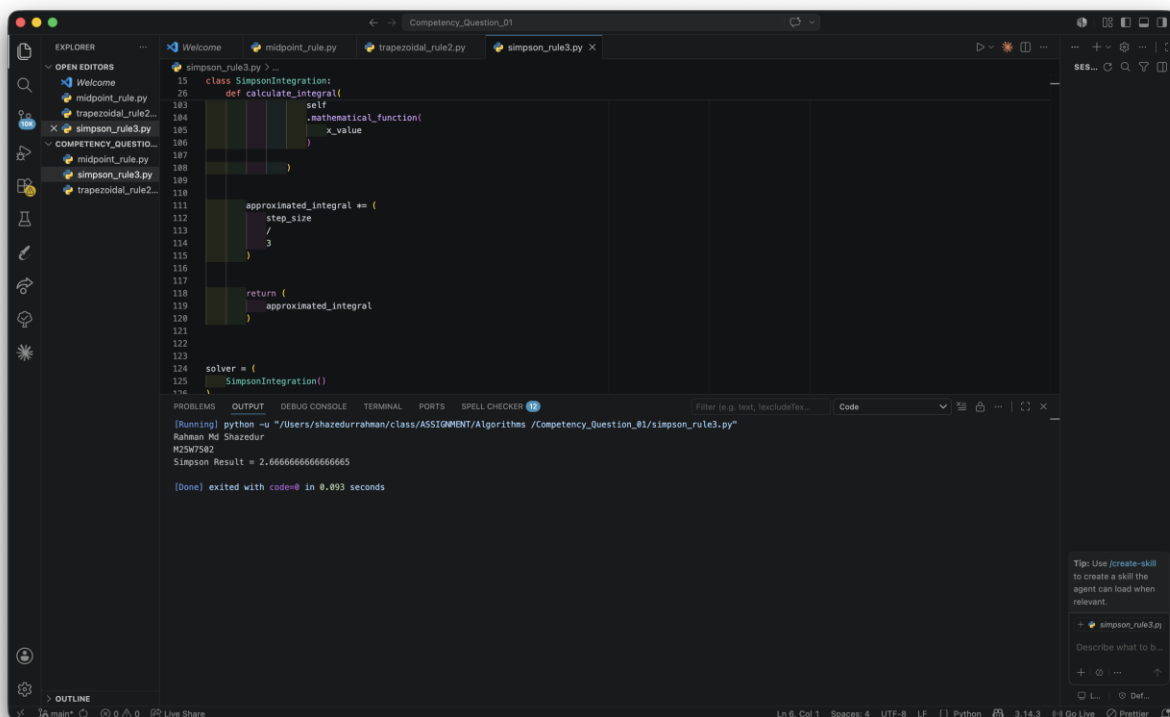
The screenshot shows the VS Code editor with the file `simpson_rule3.py` open. The code defines a `StudentInformation` class and a `SimpsonIntegration` class. The `StudentInformation` class has attributes `full_name` and `student_id`. The `SimpsonIntegration` class has a `calculate_integral` method that takes `upper_limit`, `lower_limit`, and `total_intervals` as arguments. It calculates the step size and returns the approximated integral.

```
1 # ==
2 # Student Name : Rahman Md Shazedur
3 # Student ID : M25k7582
4 # Program : Simpson Rule
5 # ==
6
7
8
9
10
11
12
13
14
15 class StudentInformation:
16
17     full_name = "Rahman Md Shazedur"
18
19     student_id = "M25k7582"
20
21
22
23
24
25
26 class SimpsonIntegration:
27
28     def mathematical_function(
29         self,
30         x
31     ):
32
33         return x * x
34
35
36     def calculate_integral(
37         self,
38         lower_limit,
39         upper_limit,
40         total_intervals
41     ):
42
43         step_size = (
44             upper_limit
45             - lower_limit
46             ) / total_intervals
47
48         approximated_integral = (
49             self
50             .mathematical_function(
51                 lower_limit
```

The screenshot shows the VS Code editor with the file `simpson_rule3.py` open. The code continues from the previous screenshot, showing the `calculate_integral` method's return statement and the `solver` and `result` variables. The `solver` variable is an instance of `SimpsonIntegration`. The `result` variable is a dictionary containing the solver and the calculated integral. The code also includes print statements for the student information and the Simpson's Rule result.

```
111         approximated_integral += (
112             step_size
113             / 3
114         )
115
116     return (
117         approximated_integral
118     )
119
120
121
122
123 solver = (
124     SimpsonIntegration()
125 )
126
127
128
129 result = {
130     solver
131     .calculate_integral(
132         4,
133         2,
134         10
135     )
136 }
137
138
139 print(
140     StudentInformation.full_name
141 )
142
143 print(
144     StudentInformation.student_id
145 )
146
147 print(
148     "Simpson Result =",
149     result
150 )
```

Output



```
15 class SimpsonIntegration:
16     def calculate_integral(self):
17         self
18         .mathematical_function(
19             x_value
20         )
21
22         approximated_integral = (
23             step_size
24             /
25             3
26         )
27
28         return (
29             approximated_integral
30         )
31
32 solver = (
33     SimpsonIntegration()
34 )
35
36 [Running] python -u "/Users/shazedurrahman/class/ASSIGNMENT/Algorithms /Competency_Question_01/simpson_rule3.py"
37 Rahman Md Shazedur
38 #3207502
39 Simpson Result = 2.6666666666666665
40
41 [Done] exited with code=0 in 0.093 seconds
```

Conclusion – Program 3

This program was developed using Simpson's Rule in Python.

The method combined weighted interval values to generate a more accurate approximation of the integral.

From this implementation, I gained practical understanding of numerical integration, algorithm execution, and problem solving.

Among the implemented methods, Simpson Rule produced the most accurate approximation result.

CODE

Midpoint Rule

```
# =====
```

```
# Student Name : Rahman Md Shazedur
```

```
# Student ID : M25W7502
```

```
# Course : Fundamentals of Algorithms
```

```
# Program : Midpoint Rule
```

```
# =====
```

```
class StudentInformation:
```

```
    full_name = "Rahman Md Shazedur"
```

```
    student_id = "M25W7502"
```

```
class MidpointIntegration:
```

```
    def mathematical_function(
```

```
        self,
```

```
        x
```

```
    ):
```

```
        return x * x
```

```
    def calculate_integral(
```

```
        self,
```

```
        lower_limit,
```

```
        upper_limit,
```

```
        total_intervals
```

):

```
step_size = (  
upper_limit  
-  
lower_limit  
) / total_intervals
```

```
approximated_integral = 0
```

```
for interval_position in range(  
total_intervals  
):
```

```
midpoint_position = (  

```

```
lower_limit
```

```
+
```

```
(  
interval_position
```


+

0.5

)

*

step_size

)

approximated_integral += (

self

.mathematical_function(

midpoint_position

)

)

approximated_integral *= (

step_size

)

return (

```
    approximated_integral
```

```
)
```

```
midpoint_solver = (
```

```
MidpointIntegration()
```

```
)
```

```
final_result = (
```

```
midpoint_solver
```

```
.calculate_integral(
```

```
0,
```

```
2,
```

```
10
```

```
)
```

```
)
```

```
print(
```

```
StudentInformation.full_name
```

```
)
```

```
print(
```

```
StudentInformation.student_id
```

```
)
```

```
print(
```

```
"Midpoint Result =",
```

```
final_result
```

```
)
```

Trapezoidal Rule

```
# =====
```

```
# Student Name : Rahman Md Shazedur
```

```
# Student ID : M25W7502
```

```
# Program : Trapezoidal Rule
```

```
# =====
```

```
class StudentInformation:
```

```
    full_name = "Rahman Md Shazedur"
```

```
    student_id = "M25W7502"
```

```
class TrapezoidalIntegration:
```

```
    def mathematical_function(
```

```
        self,
```

```
        x
```

```
    ):
```

```
        return x * x
```

```
    def calculate_integral(
```

```
        self,
```

```
        lower_limit,
```

```
        upper_limit,
```

```
        total_intervals
```

```
    ):
```

```
step_size = (  
    upper_limit  
    -  
    lower_limit  
    ) / total_intervals
```

```
approximated_integral = (
```

```
    self  
    .mathematical_function(  
        lower_limit  
    )
```

```
    +
```

```
    self  
    .mathematical_function(  
        upper_limit  
    )
```

```
)/ 2
```

```
for interval_position in range(
```

```
1,  
total_intervals  
):
```

```
x_value = (  
lower_limit  
+  
interval_position  
*  
step_size  
)
```

```
approximated_integral += (
```

```
self  
.mathematical_function(  
x_value  
)
```

```
)
```

```
approximated_integral *= (  
step_size  
)
```

```
return (  
    approximated_integral  
)
```

```
solver = (  
    TrapezoidalIntegration()  
)
```

```
result = (  
    solver  
    .calculate_integral(  
        0,  
        2,  
        10  
    )  
)
```

```
print(  
    StudentInformation.full_name  
)
```

```
print(  
    StudentInformation.student_id  
)
```

```
print(  
    "Trapezoidal Result =",  
    result  
)
```

Simpson Rule

```
# ===  
# Student Name : Rahman Md Shazedur  
# Student ID : M25W7502  
# Program : Simpson Rule  
# ===
```

```
class StudentInformation:
```

```
    full_name = "Rahman Md Shazedur"
```



```
student_id = "M25W7502"
```

```
class SimpsonIntegration:
```

```
def mathematical_function(  
    self,  
    x  
):
```

```
    return x * x
```

```
def calculate_integral(  
    self,  
    lower_limit,  
    upper_limit,  
    total_intervals
```

```
):
```

```
step_size = (  
    upper_limit  
    -  
    lower_limit  
    ) / total_intervals
```

```
approximated_integral = (
```

```
    self  
    .mathematical_function(  
        lower_limit  
    )
```

```
    +
```

```
    self  
    .mathematical_function(  
        upper_limit  
    )
```

```
)
```

```
for interval_position in range(  
    1,  
    total_intervals
```

):

```
x_value = (  
lower_limit  
+  
interval_position  
*  
step_size  
)
```

```
if (  
interval_position  
%  
2  
==  
0  
):
```

```
approximated_integral += (
```

```
2
```

```
*
```

```
self
```

```
.mathematical_function(  
x_value
```

)

)

else:

approximated_integral += (

4

*

self

.mathematical_function(

x_value

)

)

approximated_integral *= (

step_size

/

3

)

return (

approximated_integral

```
)
```

```
solver = (  
    SimpsonIntegration()  
)
```

```
result = (  
    solver  
    .calculate_integral(  
        0,  
        2,  
        10  
    )  
)
```

```
print(  
    StudentInformation.full_name  
)
```

```
print(  
    StudentInformation.student_id  
)
```

```
print(  
    "Simpson Result =",  
    result  
)
```